

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - VII

Roll No. :	Name:
Class :	Batch :
Date of Assignment :	Date/Time of Submission :

Hash Table:

Write a program to:

- a. Implement a hash table with separate chaining for collision handling.

Code:

```
size = 5
table = [ [ ] for _ in range(size)]

def insert (key,value):
    table [(key%size)].append((key,value))

insert(10,"Shiv")
insert(15,"Shiva")
insert(20,"Shivaya")
insert(32,"Shivam")
insert(33," Shree")

print(table)
print("\nAmit_S057")
```

Output:

```
===== RESTART: C:/Users/mvlui/Desktop/Amit_S057/table.py =====
[[ (10, 'Shiv'), (15, 'Shiva'), (20, 'Shivaya')], [], [(32, 'Shivam')], [(33, ' Shree')], []]
Amit_S057
|
```

- b. Store and retrieve data from the hash table.

Code:

```
size = 5
table = [[] for _ in range(size)]

def insert(key, value):
    table[key % size].append((key, value))

def search(key):
    for k, v in table[key % size]:
        if k == key:
            return v
    return "Not Found"

insert(10, "Shiv")
insert(15, "Shiva")
insert(20, "Shivaya")
insert(32, "Shivam")
insert(33, "Shree")

print(table)

key = int(input("Enter Key: "))
print(search(key))

print("\nAmit_S057")
```

Output:

```
===== RESTART: C:/Users/mvlui/Desktop/Amit_S057/table.py =====
[[ (10, 'Shiv'), (15, 'Shiva'), (20, 'Shivaya') ], [], [(32, 'Shivam')], [(33, 'Shree')], []]
Enter Key: 33
Shree

Amit_S057

===== RESTART: C:/Users/mvlui/Desktop/Amit_S057/table.py =====
[[ (10, 'Shiv'), (15, 'Shiva'), (20, 'Shivaya') ], [], [(32, 'Shivam')], [(33, 'Shree')], []]
Enter Key: 35
Not Found

Amit_S057
|
```

Curiosity Questions:

1. In a BST, which side (left or right) has **smaller numbers** than the root?
2. If we insert numbers 10, 20, 5 into a BST, which number will be at the root?
3. What will the inorder traversal of a BST give — sorted numbers or random numbers?
4. Where will the **largest number** in a BST always be found?
5. If the root of a BST is 50 and we search for 70, will we go left or right? Why?
6. If a BST has only **one node**, what is its inorder traversal?
7. Can we create a BST from characters like A, B, C instead of numbers?
8. If we insert numbers 5, 10, 15, 20 in order, will the tree be balanced or like a straight line?
9. In a BST, do we check the left child first or the right child first in inorder traversal?
10. If we delete the root of a BST, will the tree disappear?